Design Decision Record

The Governed Data Product for AI -- Berka (PKDD'99) bank data

Context. Turn seven raw banking tables (1.06 m transactions, 5,369 clients, 4,500 accounts, 682 loans) into a trusted, documented, entitlement-aware data product that an AI assistant can query correctly, and answer ten business questions through it. The brief warns that time spent on infrastructure is time misread, so the stack is one database, numbered SQL files, plain Python and a Makefile: `make up` starts it, `make all` runs it end to end in about six minutes.

Decisions

1. PostgreSQL 16, with the pipeline as SQL files plus Python orchestration. *Options:* a cloud warehouse with dbt and an orchestrator; DuckDB in-process; PostgreSQL. *Choice:* PostgreSQL. *Why:* Task 3 is enforcement, and the only honest way to show that an LLM-generated query cannot bypass an entitlement filter is to put the filter in the engine and prove it by connecting as a restricted user. DuckDB has no row level security, so that task would have become prose and untested SQL. A warehouse would also have contradicted the bank's own direction of travel, which is back on-premises for data sovereignty. *Cost:* no built-in lineage or scheduling, and a single node. Both are acceptable at this volume and neither is on the critical path of the brief.

2. Star schema with a bridge table, plus three curated views. *Options:* one big table flattened onto the transaction; Data Vault; keep the source model. *Choice:* star schema -- four dimensions, one bridge, two base facts, one derived fact. *Why:* the ten questions have four different grains (client, account, loan, posting). One big table at posting grain turns every count of clients, accounts and loans into `count(distinct ...)`, which is exactly the construct generated SQL gets wrong quietly. Data Vault is the right answer for integrating many sources with history; there is one source and a single 1999 extract, so it would triple the object count before a single question could be answered. The source model hides its meaning in Czech codes and in a disposition table whose `type` column decides who owns an account -- deciding that once is the modelling work. *Cost:* joins, and a derived month-end balance fact that has to be rebuilt. The curated views give the model a join-free surface anyway.

3. Three severities, with a mechanical consequence each. *Options:* reject every defective row; repair silently; grade the defects. *Choice:* grade them -- `block` quarantines the row and keeps it out of the product, `warn` keeps the row and attaches a flag that travels into the model and into the semantic layer, `info` standardises and logs. *Why:* a severity without a consequence is a comment. The clearest case in this data: 14 transactions have an amount of exactly zero. The naive rule ("amount must be positive") is defensible and, at `block`, would quarantine 14 valid interest postings and break 14 accounts' balance chains to fix nothing -- so it is `info`. Conversely one duplicate transaction by business key is `block`: same account, date, type, amount and resulting balance as another posting, which no primary key can catch and which double counts a flow. *Cost:* three code paths instead of one, and a quarantine table to keep. In exchange, every exclusion is reversible, countable and attributable to a named rule.

4. The semantic layer is a validated artefact, not prompt text. *Options:* describe the schema in the prompt; use a BI tool's metrics layer; write the semantics as a file the pipeline checks. *Choice:* a YAML file -- entities with their grain, certified metric expressions, a glossary that says what each term is *not*, and the data quality caveats with row counts -- which the build validates against the live database. A described column that does not exist, an existing column nobody described, or a declared grain that is not actually one row per key all fail the build. *Why:* a semantic layer that has drifted from the schema is worse than none, because it gives the model confident wrong information and the model cannot check it. *Cost:* the layer has to be maintained alongside the model; the validator is what makes that safe. *Result:* the model answered 10 of 10 questions on the first attempt, matching all ten hand-written reference answers, with no retry and no SQL hand-corrected.

5. Entitlements are row level security on the base tables, keyed on `session_user`. *Options:* instruct the model to add the filter; filter in an application gateway; policies in the database. *Choice:* policies, with the gateway's SQL guard kept as a second line. *Why:* asking the model to filter itself is a request, not a control -- the thing being filtered is the thing being asked to apply the filter. A gateway is real but one unanticipated query shape defeats it. A policy sits inside the plan of every statement on the connection; the model only ever emits text, and the predicate is not in the text, so rewriting the text cannot remove it. Identity comes from the authenticated database role rather than a session variable, because a variable can be set by anything that can send SQL. Nine bypass attempts are in the test suite and all nine fail. Unrestricted access is modelled as a portfolio containing every client, not as an exemption: the bank-wide user has `rolbypassrls = false` and goes through the policies like everyone else. Onboarding a user or a portfolio is four inserts and one credential -- no policy, view or query changes. *Cost:* a database role per user, which conflicts with naive connection pooling; the practical answers are a pool per role or `SET ROLE` performed by a trusted gateway on a privileged connection, where the gateway and not the model picks the role.

6. The LLM is treated as an untrusted query author. It is called with no tools, no MCP servers, no filesystem and no database access, from a working directory outside the repository: it sees the rendered semantic layer and the question, and nothing else. Its SQL is parsed with sqlglot -- not pattern-matched -- and must be a single read-only statement on the `ai` schema, with a row cap and a statement timeout applied. *Why parse rather than match:* `SELECT 1 FROM ai.client/**/;/**/DELETE FROM core.fact_loan` is one line to a regular expression and three statements to a parser. *Cost:* a parser dependency, and legitimate queries are occasionally stricter than necessary (every table must be schema-qualified).

What breaks at 100,000x the volume

At 100,000x this is 100 billion postings, 450 million accounts and 537 million clients -- roughly 10 TB of fact data before indexes. Four things break, in this order:

1. **The full-rebuild pipeline.** `core` is built with `CREATE TABLE AS` from

staging on every run. At 10 TB that is hours of rewriting data that did not change. It becomes an incremental load: range-partition the transaction fact by month, rebuild only the open partition, and merge rather than replace. The layer contract and the SQL stay as they are; the write pattern changes.

2. **The data quality runner.** 123 rules, each a full scan, with violating

rows materialised in Python to be sampled. The scans have to become partition-scoped (check what arrived, not what was already certified), the violating keys have to be written by `INSERT ... SELECT` instead of travelling through Python, and the aggregate rules (the per-account balance reconciliation) need incremental state per account rather than a recomputation over all history. Row-level `info` rules move to sampling with a published confidence interval; `block` rules stay exact, because quarantining is a decision about a specific row.

3. **The month-end balance fact.** 450 m accounts x 72 month ends is 32 billion

pre-computed rows, and it is rebuilt from scratch today. It becomes an incrementally maintained snapshot: one new partition per month end, computed from the previous snapshot plus that month's postings, which is O(new postings) instead of O(all history).

4. **The entitlement predicate.** `account_id IN (SELECT ... FROM sec.v_my_accounts)`

is a cheap hash join over 4,500 accounts and a large one when a portfolio holds millions. The entitlement resolution becomes a materialised, indexed table keyed `(role, account_id)`, refreshed on change and co-partitioned with the fact so the filter is a partition-local semi-join. The *design* does not change -- the predicate is still a policy on the table, and onboarding is still an insert.

Two things do **not** break, and they are the parts worth keeping: the semantic layer (a few thousand tokens regardless of volume, and the thing that decides whether the model is right) and the place the filter sits (in the engine, not in the prompt). What would change about the engine is the engine: a single PostgreSQL node is the wrong host for 10 TB, and the on-premises choices worth evaluating are PostgreSQL with Citus for scale-out, or a columnar MPP engine such as ClickHouse or StarRocks, with the same schemas, the same rule set and the same policy model ported onto it. The decision to keep the stack small is what makes that port a week of work rather than a rewrite.

Generated by `make ddr` at 2026-10-22T10:47:18+00:00 (git 95de493). Source text: deliverables/DDR_source.md